

Computational Logic

Microsoft Teams platform – **Logic IE, 2025-2026** team, code: **s9gtl7b**

Objectives:

The aim of the course is the presentation of the logical foundations of computer science:

- ◆ propositional calculus and predicate calculus, theorem proving methods,
- ◆ Boolean algebras and Boolean functions, logic circuits.

Additionally, notions related to numbers representation are introduced.

Competencies

- to use numeration systems and internal numbers representations;
- to understand the theoretical aspects of classical logics;
- to understand classical logics from a computational perspective (apply specific proof methods);
- to model human reasoning and mathematical reasoning using propositional logic and predicate logic;
- to understand logic circuits and to simplify them using specific simplification methods for Boolean functions.

Methods

- **lectures, exercises, individual study;**
- **specific bibliographic materials are used:**
books, articles, internet resources;
- **individual homework**
- **optional homework**

Content

I. Numeration systems, numbers representations

==> prerequisite for the discipline: *Computer Architecture*

II. Classical logics: propositional and predicate logics

==> prerequisite for the disciplines:

*Logic programming, Artificial Intelligence,
Automated theorem proving systems.*

III. Boolean algebras, Boolean functions and logic circuits

Evaluation - share in the final grade (%)

- **Midterm exam** (written exam in the sixth week during the seminar): **20%**
 - subjects from:
 - ❖ operations and conversions in different numeration bases
 - ❖ internal representations of integer and real numbers
 - grade ≥ 5
- **Seminars' activity**: **20%**
 - individual homework and presentation of the solved exercises during seminars
 - subjects from propositional and predicate logics and circuits
 - grade ≥ 5

Evaluation - share in the final grade (%)

- **Final exam** (theory and exercises): **60%**
 - **written paper (during the exam session)**
 - **subjects from: propositional logic, predicate logic, logic circuits**
 - **grade ≥ 4.5**
- **Optional homework**: **10%** (increases the final grade)
 - Solve/propose a set of exercises that model mathematical reasoning and common-sense reasoning, using the studied proof methods for propositional and predicate logics.



Rules for seminars

- **Attendance at the seminars is mandatory for**
- **at least 75% (minimum 10.5 seminars out of 14)**

IMPORTANT

- It is not allowed to register the didactic activities!
- According to LEN 2011, the registration by any procedure of the didactic activity can be done only with the consent of the teacher.

I. Numeration systems, numbers representations

❖ Numeration systems

- definitions, **representation** and **operations** (addition, subtraction, multiplication, division) of numbers in a base b .
- **conversions** between two bases, using three methods, for integer and rational numbers.

❖ Numbers' representation

- representation for unsigned integers, operations.
- **codes (direct, inverse, complementary)** for signed integers and subunitary numbers.
- representations of real numbers:
fixed-point representation, **floating-point** representation.

References

1. F. Boian: ***Bazele Matematice ale Calculatoarelor***, Editura Presa Universitara Clujeana, 2002.
2. M. Cocan, B. Pop: ***Bazele matematice ale sistemelor de calcul (chapter 1)***, Editura Albastra, Cluj-Napoca, 2001.
3. A. Vancea, F. Boian, D. Bufnea, A. Gog, A. Darabant, A. Sabau: ***Arhitectura calculatoarelor. Limbajul de asamblare 80x86, (chapter 1)***, Editura Risoprint, Cluj-Napoca, 2005.

II. Classical logics: propositional logic and predicate logic

- ◆ Propositional calculus and predicate calculus are presented from an **algebraic point of view** and as **deductive systems (computational perspective)**.
- ◆ Theorem proving methods are used to decide whether a statement (*conjecture*) is a **logical consequence** of a set of statements (*axioms* and *hypotheses*).
- ◆ Aim: formalization of human and mathematical reasoning using these classical logics.

LOGIC

LOGIC:

the science that studies the principles of reasoning and valid inference.

Philosophical logic and mathematical logic are commonly associated with deductive reasoning, which determines whether the **truth of a *conclusion* can be obtained for an **inference (deductive) rule**, based solely on the **truth** of the *set of premises (hypotheses)*.**

IMPORTANCE OF LOGIC

Logic is all about how you go through the world making decisions and solving problems. Wonder why logic is so important? Here are some reasons:



- 1** Helps With Decision-making
- 2** Needed At Every Step Of Your Daily Life
- 3** Makes You Better In Your Professional Life
- 4** Gives You An Edge Over Others
- 5** Make You More Confident
- 6** Makes You Better At Persuasion
- 7** Needed For Problem Solving
- 8** Makes You A Good Citizen
- 9** Helps You Understand Basic Fallacies
- 10** Keeps Stress Away
- 11** Helps Decrease An Overly Emotional Response
- 12** Increases Your Interpersonal Skills

For more information,
please visit:

www.curiousdesire.com

Timeline of research in logic

- 450 B.C. Stoics - **propositional logic** (PL), **inference**
- 322 B.C. Aristotle - quantifiers, **syllogism rule**: dominant model of correct argumentation
- 1646 -1716 Leibniz – the founder of **symbolic logic**, he proposed a **general decision procedure** to check the validity of formulas
- 1847 George Boole – algebras, **formalization of propositional logic**
- 1847 Augustus De Morgan: **formal logic**
- 1879 Gottlob Frege – **predicate or first-order logic** (FOL)
- 1889 Peano - 9 axioms for natural numbers
- 1920 Hilbert's program
- 1921 Emil Post- **truth tables**, 1922 Wittgenstein - **proof by truth tables**

Timeline of research in logic (contd.)

- 1929 Gödel completeness theorem of FOL
- 1930 Herbrand - a ***proof procedure for FOL*** based on propositionalization
- 1931 Gödel incompleteness theorems for the consistency of Peano axioms
- 1936 Gentzen - a proof for the consistency of Peano axioms in set theory
- 1936 Church and Turing: ***undecidability of FOL***
- 1954 Davis - ***first machine-generated proof***
- 1955 Beth - ***Semantic Tableaux***
- 1957 Newell - First machine-generated proof in Logic Calculus
- 1958 Prawitz - First prover for FOL
- 1959 Gilmore, Wang - more provers
- 1960 Davis - Putnam ***Procedure***
- 1963 Robinson - ***Unification, resolution***
- 1968 R.Smullyan – ***Semantic tableaux proof method***

Propositional and predicate logics

1. **Syntax:** connectives, quantifiers, terms, atoms, clause, literal, formulas.
2. **The formal system** associated to propositional/predicate logic.
3. **Semantics of propositional/predicate logic:** interpretation, model, valid/consistent/inconsistent formula, logical consequence.
4. **Normal forms in propositional and predicate logics.**
5. **Theorem of soundness, completeness for propositional/predicate logic; non-contradiction, coherence and decidability of propositional logic, semi-decidability of predicate logic.**
6. **Semantic tableaux method** – a semantic and refutation proof method.
7. **Resolution method** – a direct and refutation proof method.
8. **Formalization of common-sense and mathematical reasoning using propositional and predicate logic.**

Propositional logic

Logical propositions are models of propositional assertions from natural language, which can be **true** or **false**.

P: It is sunny.

Q: It is hot outside.

R: I go to the swimming pool.

S: *if* it is sunny *and* it is hot outside *then*

I will go to the swimming pool.

S: $P \wedge Q \rightarrow R$

“Theorem proving”:

From P,Q,S (hypotheses) can we deduce (infer) R (conjecture)?

Predicate (first-order) logic

The axioms that define the natural numbers:

a1. *Every* natural number has exactly one immediate successor.

existence: $(\forall x)(\exists y)equal(y, successor(x))$

unique: $(\forall x)(\forall y)(\forall z)(equal(y, successor(x)) \wedge equal(z, successor(x)) \rightarrow equal(y, z))$

a2. The natural number 0 is *not* the immediate successor of any natural number.

$\neg(\exists x)equal(0, successor(x))$

a3. *Every* natural number except 0 has exactly one immediate predecessor.

$(\forall x)(\exists y)(\neg equal(0, x) \wedge equal(y, predecessor(x)))$

Functions: *successor, predecessor*; Predicates: *equal*

Modelling reasoning using predicate logic

Hypotheses:

H1. If x is perpendicular to y then x intersects y .

H2. If x is parallel to y then x does not intersect y .

H3. If x is perpendicular to y and z is perpendicular to y
then x is parallel to z .

H4. $d1$ is perpendicular to d .

H5. d is perpendicular to $d2$.

Conclusion: C. $d1$ does not intersect $d2$.

Check whether the conclusion C is derivable from the set of hypotheses.

Succession to the British throne – common-sense human reasoning

Hypotheses:

H1: If x is the king and y is his oldest son, then y can become the king.

H2: If x is the king and y defeats x , then y will become the king.

H3: *Richard III* is the king.

H4: *Henry VII* defeated *Richard III*.

H5: *Henry VIII* is *Henry VII*'s oldest son.

Conclusion C: Can *Henry VIII* become the king?

Types of logics

- ◆ ***classical* logics:**
 - propositional* logic**
 - predicate (first order) logic***
- ◆ ***modal* and *temporal* logics**
- ◆ ***multivalued* and *fuzzy* logics**
- ◆ ***nonmonotonic* logics**

Nonmonotonic logics

- Default Logic, Reiter 1980

“The gods did not reveal, from the beginning,
All things to us, but in the course of time
Through seeking we may learn and know things better.
But as for certain truth, no man has known it,
Nor shall he know it, neither of the gods
Nor yet of all the things of which I speak.
For even if by chance he were to utter
The final truth he would himself not know it:
For all is but a woven web of guesses’
— Xenophanes

Xenophanes, 570 BC – 478 BC, Italy
An important pre-Socratic philosopher

Nonmonotonic reasoning is an important part of human reasoning:

- ❖ it is the process of inferring conclusions (formulas that are only plausible, not necessarily true) from incomplete information;
- ❖ adding new facts can later invalidate these conclusions;
- ❖ it is not a valid reasoning.

Automated Theorem Proving (ATP)

- ◆ deals with the development of computer programs which show that some statement (the conjecture) is a logical consequence of a set of statements (the axioms and the hypotheses).

Dedicated (educational) automated theorem provers

- 3TAP, pTAP, leanTAP, Cassandra (semantic tableaux)
- OTTER, PCPROVE, AMPHION, Jape (resolution)
- HERBY (semantic trees + Herbrand theorem)

Implementations in: PROLOG, LISP, C/C++,...

ATP systems in mathematics

- EQP. In 1933 Herbert Robbins conjectured that a particular group of axioms forms a basis for Boolean algebra, but neither he nor anyone else could prove this. The proof was found in October 10, 1996, after about 8 days of search by EQP.
- Otter - used to prove several results in quasi-groups.
- Geometry Expert has been used to obtain new results in Euclidean geometry.

ATP for software generation

- the KIDS system developed at Kestrel Institute
 - used to derive scheduling algorithms that have outperformed currently used algorithms
 - provides intuitive, high level operations for transformational development of programs from specifications.
- the AMPHION project, sponsored by NASA
 - used to determine appropriate subroutines combined to produce programs for satellite guidance.
 - by encapsulating usable functionality in software components, and then reusing those components, AMPHION can develop software in less time than human programmers.

ATP for software verification

- The *Karlsruhe Interactive Verifier (KIV)*
 - *verifies a large range of software applications: implementation of set functions, tree and graph representation and manipulation.*
 - *used for industrial application: a software controlled railway switch, safe command transfer in a space vehicle, and supervision of neutron flow in a nuclear reactor.*
- PVS is a verification system
 - *used in various applications: diagnosis and scheduling algorithms for fault tolerant architectures, and requirements specification for portions of the space shuttle flight control system.*
 - *NASA uses ATP to certify safety properties of aerospace software that has been automatically generated from high-level specifications.*

ATP for hardware verification

- **Hardware verification** is the largest industrial application of ATP.
- ***IBM, Intel, and Motorola*** are among the companies that employ ATP technology for verification.
- **ACL2 system** - tudor.andrei.pop4@stud.ubbcluj.ro used to obtain a proof of the correctness of the floating point divide code for AMD's PENTIUM-like AMD5K86 microprocessor
- **ANALYTICA** - used to verify a division circuit that implements the floating point standard of IEEE.
- **HOL system** - used at Bell Laboratories for hardware verification.

Challenges LLMs face with logic

- **Pattern-based, not truth-based:**

LLMs are trained on vast datasets and predict the next word, making them good at mimicking logical patterns but not at true logical understanding or derivation.

- **Lack of consistency:**

LLMs can generate logically inconsistent answers across different queries, failing to uphold logical rules and self-consistency.

- **Difficulty with novel problems:**

LLMs often struggle with mathematical and commonsense logic problems that deviate significantly from their training data, exposing their limitations in deriving new solutions.

How LLMs and logic are combined

- **LLM + Symbolic Solvers:**

This approach uses an LLM to transform a natural language problem into a formal, **symbolic representation that a deterministic symbolic solver** can then process to perform accurate inferences.

- **Semantic Parsing:**

An LLM can act as a few-shot semantic parser, **converting natural language sentences into a logical form that can be used by a logic-based system**, such as an answer set program (ASP).

- **Neuro-symbolic Integration:**

Some systems integrate LLMs with **traditional proof-search engines**, using the LLM to guide the search process and replace human-provided heuristics.

References

- M. Ben-Ari: *Mathematical Logic for Computer Science*, Ed. Springer, 2001
- M. Clarke: *Logic for Computer Science*, Ed. Eddison-Wesley 1990.
- M. Fitting: *First-order logic and Automated Theorem Proving*, Ed. Springer Verlag, 1990.
- Mihaela Malita, Mircea Malita: *Bazele Inteligentei Artificiale, Vol. I, Logici propozitionale*, Ed. Tehnica, Bucuresti, 1987 – library.
- M. Lupea, A. Mihiș: *Logici clasice și circuite logice. Teorie și exemple*, Editura Albastra, edition I- 2008, edition II– 2009, edition III- 2011.
- M. Lupea, A. Mihis: *A Computational Approach to Classical Logics and Circuits, Presa Universitara Clujeana, Cluj-Napoca*, edition II-2016 .
- L.C. Paulson: *Logic and Proof*, Univ. Cambridge, 2000, on-line course.
- M. Possega: *Deduction Systems*, Inst. of Informatics, 2002, on-line course.
- D.Tatar: *Bazele matematice ale calculatoarelor*, edition 1999- library.

III. Boolean algebras, Boolean functions

Logic circuits

LOGIC GATES SYMBOLS & FUNCTIONS



AND GATE

A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1



OR GATE

A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1



NOT GATE

Input	Output
0	1
1	0

III. Boolean algebras, Boolean functions

Logic circuits

- ❖ In 1938 **Claude Shannon** proved that a ***two-valued binary Boolean algebra*** can describe the operations of ***two-valued electrical switching circuits***.
- ❖ ***Propositional logic*** is used to minimize the number of gates in a circuit, and to show the equivalence of ***combinational circuits***.
- ❖ **Grigore Moisil** (1906-1973) invented the ***three-stable circuits*** and had important contributions in the fields of ***algebraic logic*** and differential equations. ***Moisil used propositional logic to minimize Boolean functions.***
- ❖ In modern times ***Boolean algebras*** and ***Boolean functions*** are indispensable in the design of computer chips and digital circuits.

III. Boolean algebras, Boolean functions and Logic circuits (contd.)

1. **Boolean algebras**: definition, properties, principle of duality, example.
2. **Boolean functions**: definitions, maxterms, minterms, canonic disjunctive/conjunctive forms.
3. **Simplification of Boolean functions**:
 - **definitions**: maximal monoms, central monoms, factorization;
 - **Veitch-Karnaugh diagrams method**
 - **Quine's analytical method, Moisil's algebraic method.**
4. **Logic circuits**
 - definitions, representations for **basic gates** ("and", "or", "not") and **derived gates** ("xor", "nand", "nor")
 - examples of logic circuits: **adder, subtractor, decoder, encoder, comparator**

References

1. M. Cocan, B. Pop: Bazele matematice ale sistemelor de calcul (chapter 2), Editura Albastra, Cluj-Napoca, 2001 – library.
2. M. Lupea, A. Mihiș: *Logici clasice și circuite logice. Teorie și exemple.*, Editura Albastra, edition I -2008, edition II - 2009, edition III - 2011.
3. M. Lupea, A. Mihis: *A Computational Approach to Classical Logics and Circuits*, Editura Universitara Clujeana, second edition, 2016.
4. D.Tatar: Bazele matematice ale calculatoarelor, edition 1999, library.